Proba: A High-Performance, Low-Traffic Probabilistic Spatial Memory Streaming Prefetcher

Yinting Huang
Huawei Newton Research Centre
Cambridge, United Kingdom
yinting_justin_huang@outlook.com

Jacky Wong
Imperial College London
London, United Kingdom
jacky.wong20@imperial.ac.uk

Sam Ainsworth
University of Edinburgh
Edinburgh, United Kingdom
sam.ainsworth@ed.ac.uk

Abstract

Spatial Memory Streaming (SMS) prefetchers exploit recurring spatial access patterns across memory regions to predict future accesses. Despite their effectiveness, existing SMS-based designs suffer from coarse control over aggressiveness and limited adaptability to dynamic program behavior and historical access patterns. In this work, we present Proba, a feedback-directed SMS-based prefetcher that integrates probabilistic pattern updates, marginal accuracy decision-making, and pattern accumulation over several trigger features to achieve fine-grained accuracy–coverage trade-offs. When accuracy deteriorates, Proba moderates its aggressiveness to prioritize accurate prefetches; conversely, when accuracy rises, it expands its coverage. Proba achieves higher speedups than any other SMS-based prefetcher while being vastly more efficient in DRAM traffic. This culminates in a large performance gap between Proba and other techniques in challenging multicore and DRAM-constrained scenarios without harming single-threaded throughput.

CCS Concepts

• **Computer systems organization** → **Architectures**; **Superscalar architectures**.

Keywords

Prefetching, Cache Systems, Microarchitecture

1 Introduction

The ever-growing gap between processor performance and memory access latency continues to throttle application performance despite extensive on-chip optimizations. Even though clock frequencies have plateaued since the early 2000s, architectural advances such as out-of-order execution, wider pipelines, and multicore scaling have steadily increased effective processor performance. In contrast, memory latency and bandwidth have improved at a much slower pace. As a result, the gap between processor speed and memory access time has widened, leading to frequent stalls and underutilization of computational resources.

To mitigate this memory wall, processors employ data prefetchers that anticipate future memory accesses and fetch data into on-chip caches before demand arises. One widely deployed prefetcher is Spatial Memory Streaming (SMS) [15, 35, 36]. SMS is built on the observation that memory accesses often exhibit spatial regularity:

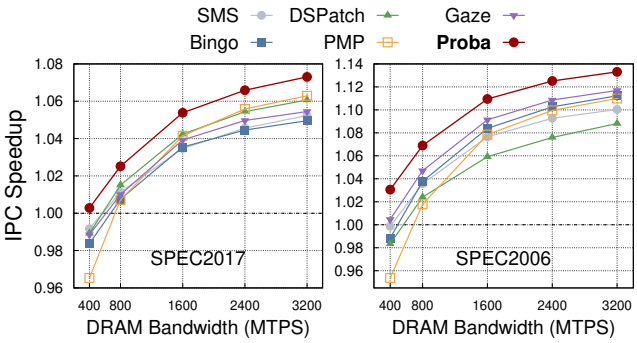


Figure 1: Geomean speedup of SMS-based prefetchers at various DRAM bandwidth constraints. Proba consistently outperforms others because it is highly frugal in traffic.

a traversal pattern observed in one memory region is likely to reappear in another. SMS encodes these access patterns in compact bit patterns, indexed by trigger features: metadata associated with the accesses that begin a spatial region, such as the program counter (PC) or the offset of the first access within the region. When the same trigger feature appears in a different region, SMS replays the prediction stored in the history table. SMS is a lightweight yet effective mechanism for capturing recurring spatial patterns and has been adopted in modern cores such as the Neoverse V2 [9].

Although SMS improves performance on many workloads, it incurs substantial memory overhead by issuing prefetches even when access behavior does not exhibit predictive patterns across regions. This excessive memory consumption reduces the efficiency of SMS and can degrade performance under bandwidth-constrained conditions. DSPatch [6] attempts to address this problem by maintaining two prediction bit patterns: one biased toward coverage and the other biased toward accuracy. At runtime, it selects between these two patterns according to observed DRAM-bandwidth utilization. While effective, this modulation mechanism remains coarse-grained, as it can only modulate between two fixed update behaviors. More recent work, PMP [21], improves on this by introducing a pattern-merging strategy based on dual pattern tables and saturating counters. PMP accumulates patterns for different trigger features in separate tables and uses saturating counters to guide which merged pattern should be used for prefetching. However, these counters are storage-intensive and reduce the effective storage capacity available for pattern history, limiting PMP to relatively coarse-grained trigger features and constraining its prediction capability. Furthermore, because both DSPatch and PMP require greater storage per entry, they must compress metadata and coarsen the

This work is licensed under a Creative Commons Attribution 4.0 International License.
PACT '26, Chicago, IL, USA
© 2026 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-2915-7/2026/10
https://doi.org/10.1145/3838684.3846832

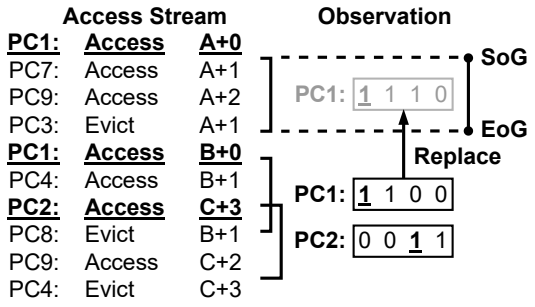


Figure 2: Basic operation of the Spatial Memory Streaming (SMS) prefetcher. SMS records the access pattern observed from the first access to a memory region until its first eviction, and reuses the recorded pattern to predict accesses when the same trigger feature is observed in another region.

granularity of pattern updates, which biases them toward higher coverage at the expense of accuracy.

We propose Proba, a novel probabilistic update mechanism for SMS-based prefetchers. Instead of interpolating between two history patterns as in DSPatch or accumulating patterns with saturating counters as in PMP, Proba consolidates alternating patterns into a single, progressively refined prediction, maintaining high accuracy without additional storage overhead. The update probability is dynamically adjusted via accuracy feedback to regulate prefetch aggressiveness. Through gradual probabilistic updates, Proba adapts smoothly to evolving program access patterns and workload characteristics, improving the performance benefit of spatial memory streaming while mitigating unnecessary DRAM traffic. We further design a *marginal accuracy* mechanism that is able to dynamically adapt predictions using the trigger features of several state-of-the-art SMS-based prefetchers [6, 11, 21, 36]. Finally, we design a Jail Table that decouples pattern discovery from region tracking, enabling precise cache-generation tracking without large storage buffers.

As shown in Figure 1, Proba consistently outperforms all state-of-the-art SMS-based prefetchers, regardless of whether DRAM bandwidth is ample or highly constrained. It limits inaccurate prefetches while remaining coverage-seeking when additional prefetching is beneficial. This becomes even more pronounced in multicore workloads, where shared memory bandwidth is scarce. Proba's frugal use of bandwidth makes it well suited to modern energy-bound systems [16, 18] across the scale from mobile phones to data centers.

2 Background and Motivation

2.1 Spatial Memory Streaming (SMS)

SMS [36] is built on the observation that commercial applications often exhibit recurring spatial patterns across memory regions. To capture these patterns, SMS divides memory into fixed-size spatial regions (e.g., 2KiB or 4KiB). The first access to a region is the trigger access, marking the Start-of-Generation (SoG). For each trigger, SMS records the trigger Program Counter (PC) along with the cache-line access offset in the Active Generation Table (AGT). Subsequent accesses within the same region are recorded by setting the corresponding bit positions in the bit pattern. Tracking of a region terminates when any of the accessed cache lines is evicted from the cache, signaling the End-of-Generation (EoG). The resulting bit pattern is then stored in the Pattern History Table (PHT) indexed by the trigger `PC+Offset`. When an access with the same trigger feature occurs in a new spatial region, SMS uses the stored bit pattern to issue prefetches. The basic mechanism of SMS is shown in Figure 2.

The baseline SMS lacks the ability to dynamically adapt prefetch aggression based on the observed memory accesses. Upon encountering a new access pattern, SMS replaces the existing pattern in the Pattern History Table (PHT) with the new observation, thereby discarding all historical information. This assumes that the most recent observation is most accurate, which is not guaranteed. Consequently, SMS oscillates between patterns and suffers from poor adaptivity.

2.2 Dual Spatial Pattern Prefetcher (DSPatch)

In response to the limitations of SMS, the Dual Spatial Pattern Prefetcher (DSPatch) [6] introduces a mechanism to dynamically balance accuracy and coverage at the pattern level. As shown in Figure 3, DSPatch maintains two modulated bit patterns with associated quantitative metrics: one biased toward accuracy (AccP), the other toward coverage (CovP). DSPatch selects a pattern based on both pattern-level metrics and system bandwidth utilization. When bandwidth utilization is high, DSPatch prioritizes accuracy-biased patterns. When bandwidth utilization is low, it prefetches the coverage-biased pattern if its accuracy is acceptable; otherwise, it issues the accuracy-biased pattern.

DSPatch uses a Page Buffer (PB) to record the trigger PC and offset together with the observed spatial access pattern. Its Signature Prediction Table (SPT) stores two modulated bit patterns per entry. Bit patterns are modulated by combining new observations with stored patterns: bitwise OR operations bias the pattern toward coverage, while bitwise AND operations bias toward accuracy. The SPT is organized as a tagless direct-mapped structure indexed only by the trigger PC, and stores bit patterns anchored to the trigger offset as discussed in the SMS variant [15]. To maintain two bit patterns without doubling the storage requirement, DSPatch compresses them by merging adjacent bits into a single bit and stores these compact representations in the SPT, allowing neighboring cache lines to be encoded compactly. The compression scheme is similar to the one shown in Figure 5.

DSPatch employs two metrics to quantify the accuracy and coverage of its modulated bit patterns at PC granularity. During SPT updates, the popcount of the stored pattern gives the number of predicted prefetches (C_{pred}), the popcount of the observed pattern gives the number of actual accesses (C_{obs}), and the popcount of their intersection gives the number of accurate prefetches (C_{acc}). Accuracy is therefore estimated as C_{acc}/C_{pred}, while coverage is estimated as C_{acc}/C_{obs}. An example is shown in Figure 4.

To preserve more history than SMS, DSPatch updates its stored patterns through repeated bitwise OR and AND operations. However, this modulation scheme is inherently unstable: repeated OR operations drive the pattern toward all ones, while repeated AND operations drive it toward all zeros. To limit this effect, DSPatch uses a 2-bit saturating counter (`ORCount`) to track the number of

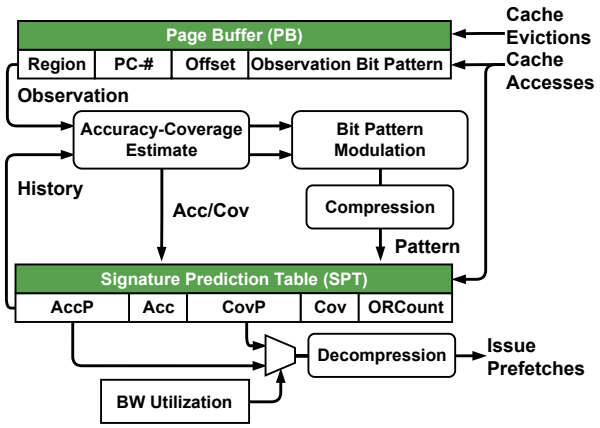


Figure 3: DSPatch tracks active pages in the Page Buffer (PB) and updates their access patterns in the Signature Prediction Table (SPT). SPT entries maintain both a coverage-seeking pattern (CovP) and an accuracy-seeking pattern (AccP).

	Bit Pattern	PopCount
Observation	1011 0100 0011 1100	**8**
Prediction	1010 0110 0000 0001	**5**
Bitwise AND	1010 0100 0000 0000	**3**

$$LocalAccuracy = \frac{C_{acc}}{C_{pred}} = \frac{3}{5}$$

$$LocalCoverage = \frac{C_{acc}}{C_{obs}} = \frac{3}{8}$$

Figure 4: An example of accuracy and coverage estimation used by DSPatch, based on the number of matching bits.

OR updates in each SPT entry. Once the counter saturates, further accumulation is stopped and the pattern is reset and retrained.

Although DSPatch tries to improve the accuracy-coverage trade-off over SMS, its effectiveness is limited by heuristics. First, repetitive OR operations cause the bit patterns to converge toward all ones. Once `ORCount` saturates, DSPatch requires retraining the entire pattern, resulting in a complete loss of historical information. At the same time, the accuracy-biased pattern is modulated from a coverage-based pattern that becomes increasingly aggressive, and can also inherit this aggressiveness. Compression further exacerbates this problem by biasing predictions toward coverage and introducing substantial inaccuracy. As shown in Figure 5 for PMP [21] where it is also used, pattern reconstruction can introduce prediction errors of up to 50%. These heuristics make DSPatch highly aggressive in all modes.

2.3 Pattern Merging Prefetcher (PMP)

The Pattern Merging Prefetcher (PMP) [21] revisits SMS from another direction. PMP (Figure 5) merges multiple patterns and uses the merged result to guide prefetching. Its key idea is that spatial regions with similar trigger features share structure, which can be exposed by accumulating observations.

Similar to SMS, PMP uses an Active Generation Table (AGT) to detect the start of a generation and record the observed spatial footprint until the region is evicted. Learned patterns are accumulated

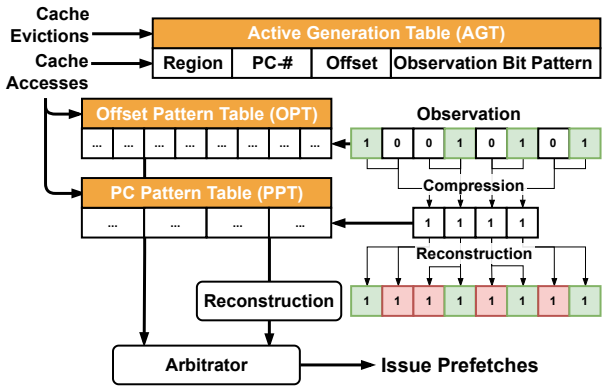


Figure 5: PMP accumulates patterns into the Offset Pattern Table (OPT) and the PC Pattern Table (PPT). The OPT stores the original pattern, while the PPT stores a compressed version. During prediction, PMP combines the OPT prediction with the reconstructed PPT prediction to issue prefetches.

into two history structures: an Offset Pattern Table (OPT), indexed by the trigger offset (`Offset-Only`), and a PC Pattern Table (PPT), indexed by the trigger PC (`PC-Only`). Both tables store counter vectors rather than plain bit patterns; the OPT keeps a fine-grained pattern, while the PPT stores a compressed version of the pattern at a coarser granularity. At prediction time, PMP extracts candidate patterns from both tables and uses arbitration over the counter vectors to determine which offsets should be prefetched and where they should be filled to.

This design improves upon SMS and DSPatch by preserving more than one recent observation and by allowing repeated patterns to reinforce one another over time. However, this comes at a storage cost. Because PMP stores confidence information as counter vectors, a substantial fraction of the hardware budget is spent on metadata instead of pattern capacity. This reduces the effective capacity for distinct histories and forces PMP to rely on coarse trigger features, namely `PC-Only` and `Offset-Only`. In contrast, other SMS-based prefetchers show that finer-grained trigger features such as `PC+Offset` and `Offset+Offset` can capture important regularity that coarse signatures miss [3, 11, 15].

PMP is also limited by the heuristics used to translate merged counters into prefetch decisions. The PPT stores a coarser pattern than the OPT, using DSPatch-like compression, and both tables rely on fixed voting thresholds to make prefetch decisions. Such coarse-grained policies do not adapt to the level of success actually attainable, where the optimal degree of prefetch aggressiveness is highly workload- and access-dependent. Consequently, PMP often generates inaccurate prefetches that increase memory bandwidth consumption, resulting in higher DRAM traffic (Figure 12).

3 Our Solution: Proba

We introduce **Proba**, an SMS-based prefetcher that leverages probabilistic updates to maintain useful patterns without requiring extra storage or lossy compression. Proba treats pattern update as a stochastic process, enabling gradual pattern evolution. Proba has the following features:

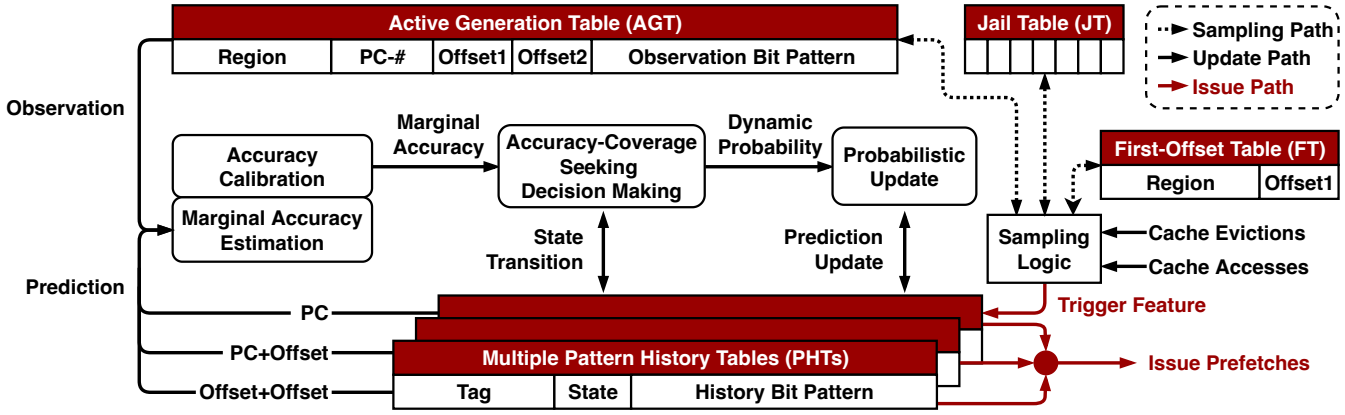


Figure 6: Design overview of Proba. Proba uses probabilistic updates to maintain accurate, high-coverage patterns in its Pattern History Tables (PHTs). Marginal accuracy dynamically controls prefetch aggression, while region sampling and Bloom-like hash-based mappings provide precise observation of cache-region generation behavior.

- **Probabilistic Update:** Stores a single bit pattern and updates it probabilistically upon each observation, eliminating both the storage overhead and the pattern corruption inherent in prior designs (Section 3.2).
- **Accuracy-Coverage Seeking Decision-making:** Incorporates accuracy feedback to drive probability state transitions, enabling fine-grained adaptive accuracy–coverage trade-offs based on runtime behavior (Section 3.3).
- **Multiple Pattern History Tables with Marginal Accuracy Decision-making:** Proba takes the union [21] of predictions of multiple PHTs indexed by distinct trigger features [6, 11, 15, 35]. Rather than assigning aggressiveness to each table in isolation, Proba adjusts each table according to its *marginal accuracy*: the accuracy of the additional prefetches it contributes beyond those already predicted by more accurate tables (Section 3.4).
- **Jail Table (JT) and Region Sampling:** Prevents resource thrashing during pattern learning by decoupling the functionality of the Active Generation Table (AGT) into two parts: a Jail Table, which tracks active spatial regions currently in the cache, and a sampling-based AGT that selectively monitors representative regions. This separation allows for precise pattern learning until the intended End-of-Generation (EoG) event when an element of the region is evicted from the cache (Section 3.5).

3.1 Overview

Proba's design is given in Figure 6. It inherits the overall organization of prior SMS-based prefetchers. Without loss of generality, we refer to the structure that tracks access patterns within a spatial region as the Active Generation Table (AGT), and the structure that stores historical metadata governing prefetch decisions as the Pattern History Table (PHT). Proba observes misses and prefetch tag-hits[1] at the L2, along with evictions in order to track generations of accesses to 4KiB regions. The AGT records the access pattern and trigger feature for each active region. When the AGT finishes tracking a region and produces a new observation, Proba estimates accuracy by comparing the observed bit pattern with the history patterns stored in the PHT. Based on this accuracy estimate, Proba performs probability state transitions and generates a probability pair. Using this pair, Proba probabilistically updates the history pattern based on new observations, enabling fine-grained pattern evolution without storage overhead. This update policy is applied across multiple trigger-feature-indexed PHTs, using marginal accuracy to ensure each trigger feature is only as aggressive as justified by its unique contribution. To avoid imprecise pattern learning and resource thrashing, Proba introduces a sampling-based AGT that selectively trains on representative regions, while a separate Jail Table tracks active regions currently present in the cache and ensures that prefetches are only triggered by valid access events. When an access with the same trigger feature occurs in a new region, Proba indexes the PHTs using various trigger features and issues prefetches from the union of their stored history patterns.

[1] As in Triangel [2] and Berti [30], Proba trains on misses and prefetch tag hits rather than all demand accesses. Where relevant we use the tag-based accuracy definitions from Feedback-Directed Prefetching [37]: A prefetched cache line has a tag associated with it in the cache. On first use, the tag is unset and the prefetch is treated as accurate. If the line is evicted while the tag remains set, the prefetch is treated as inaccurate.

3.2 Probabilistic Update

When the Active Generation Table (AGT) finishes tracking a spatial region, it produces an observation bit pattern $O \in 0, 1^L$, where L is the number of cache lines in a spatial region (i.e., the region size divided by the cache-line size), and each bit indicates whether the corresponding cache line in the region was accessed. The corresponding Pattern History Table (PHT) entries, indexed by trigger feature (Section 3.4), maintain history bit patterns $P \in 0, 1^L$. As illustrated in Figure 7, Proba treats the update of a history pattern P with observation O as a stochastic process that interpolates between the most conservative (bitwise AND) and the most aggressive (bitwise OR) behavior. For each bit position $i \in 1, \ldots, L$, Proba applies the following update rule:

$$P_i^{\text{new}} = \begin{cases} 1, & \text{with probability } p \quad \text{if } O_i = 1 \wedge P_i = 0, \\ 0, & \text{with probability } q \quad \text{if } O_i = 0 \wedge P_i = 1, \\ P_i, & \text{otherwise.} \end{cases} \tag{1}$$

where $p \in [0.0, 1.0]$ is the *insertion probability* which controls how aggressively newly observed bits are incorporated into the history,

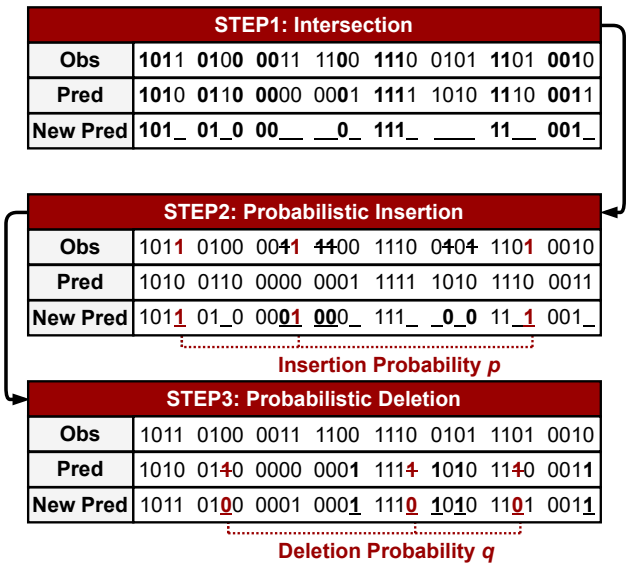


Figure 7: Probabilistic update of the Pattern History Table (PHT). Step 1 retains the intersection between the new observation (from the AGT) and the previous prediction (from the PHT). Step 2 inserts bits present in the observation but absent in the old prediction with an insertion probability p. Step 3 deletes bits present in the old prediction but missing in the new observation with a deletion probability q.

and $q \in [0.0, 1.0]$ is the *deletion probability* which controls how aggressively bits absent in the current observation are removed.

Choosing a low *insertion probability* p and a high *deletion probability* q biases the update toward accuracy by retaining only persistently observed bits; conversely, a high p and low q biases toward coverage by quickly admitting new bits and slowly forgetting stale ones. To prevent bit patterns from tending to all 1s or 0s, Proba avoids the extreme combinations $p = 1.0$, $q = 0.0$ and $p = 0.0$, $q = 1.0$. Over repeated observations, applying Eq. (1) yields an aggregated self-refreshing history pattern where fresh observations O are blended into history pattern P while obsolete patterns gradually decay. The probabilistic update[2] logic is entirely off the prefetch-issue and L2-access critical paths: randomness is required only when committing a completed observation to the PHT.

3.3 Accuracy-Coverage Seeking Decision-making

Proba dynamically adjusts its accuracy-coverage seeking behavior in response to accuracy feedback. An accuracy threshold *AccThr*, with a default value of 80%, guides adaptation. On each PHT update, if the measured accuracy exceeds *AccThr*, the prefetcher transitions toward a more coverage-seeking behavior on the next update; otherwise it shifts toward a more accuracy-seeking behavior on the next update.

Proba adopts the PHT-entry-local accuracy metric from DSPatch, which we denote as *LocalAcc*. This metric compares the new observation (O) with the history pattern used for prediction (P) to quantify correctness of the pattern.

$$LocalAcc = \frac{C_{acc} - 1}{C_{pred} - 1} \tag{2}$$

where $C_{acc} = |O \cap P|$ and $C_{pred} = |P|$, and $|X|$ is the number of elements in set X. The trigger access is excluded.

Proba encodes its accuracy–coverage decision in a 3-bit field, `State`, within each PHT entry, where each probability state corresponds to a distinct pair of insertion and deletion probabilities. The state transition logic and probability map are given in Figure 8. The mid-point, where $p = q = 1.0$, matches the behavior of the baseline SMS, and was chosen as the best-performing unbiased probability state: lower probabilities without bias, e.g. $p = q = 0.5$, performed less well. For the same reason, all coverage-seeking states use $p = 1.0$ and all accuracy-seeking states use $q = 1.0$. The most accuracy-seeking state has a lower insertion probability than the most coverage-seeking state's deletion probability. This design ensures that PHT indices (e.g. PCs) poorly covered by spatial memory streaming are not overfetched, while still allowing the prefetcher to quickly adapt and maintain a small chance of generating prefetches if their behavior changes in the future. The remaining intermediate states fill the probability space between these extremes; we tuned them on SPECspeed 2017 and found that, outside extreme settings, their exact values have little performance impact.

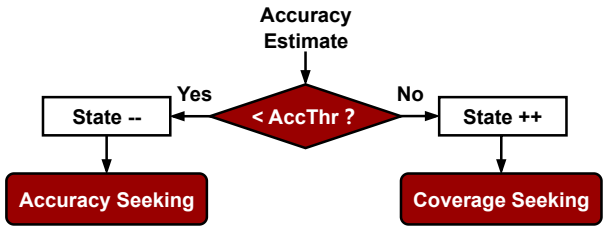
3.4 Pattern History Tables and Marginal Accuracy

Proba employs multiple Pattern History Tables (PHTs), each indexed by a distinct trigger feature, inspired by prior work [3, 6, 11, 36]. The first table is indexed by trigger PC (**`PC-Only`**) using rotated indexing similar to the SMS variant [15] and DSPatch [6]. The second table is indexed by the trigger PC combined with the trigger offset (**`PC+Offset`**), like Bingo [3] and the original SMS [36], to capture patterns where the initial access offset is informative. The third table is indexed by the temporal correlation between the first and second offsets accessed within a region (**`Offset+Offset`**), inspired by Gaze [11]. To support this, Proba uses a small First-Offset Table (FT)[3] to record the first offset observed for a region. When a distinct second offset is later accessed in the same region, the pair of offsets is used to trigger `Offset+Offset` prefetching. At prefetch time, all PHTs are queried using the corresponding trigger features. Similar to PMP [21], Proba issues prefetches from the union of their predictions.

While combining multiple trigger features in this way improves coverage, it can also reduce accuracy because some trigger features

[2]Probabilistic PHT updates occur only when an AGT observation completes, at EoG or on a premature AGT eviction (Section 3.5), and can therefore be buffered without delaying prefetch generation. Moreover, Proba requires only simple randomized update logic: a lightweight pseudo-random generator, such as a linear congruential generator, is sufficient. Randomized decisions are also common in practical cache-management mechanisms, including Bimodal RRIP (BRRIP) [20]. We tried several random number generators during our experiments, including `std::rand`, `std::minstd_rand`, `std::mt19937`, and `std::knuth_b`, and changing the RNG produced no material difference in any simulated outcome in any simulated workload.

[3]The First-Offset Table (FT) stores the first accessed offset for the last 64 regions that reached the SoG event, regardless of whether they were sampled by the AGT or placed in the Jail on the first access to the region (Section 3.5). If a second offset is accessed within that region before the region reaches EoG and before it is evicted from the FT due to capacity pressure, we trigger `Offset+Offset` prefetching. For sampled regions that update the PHT, the AGT also specifies the second offset accessed rather than just the first.



	Accuracy Seeking				Coverage Seeking			
State	**0**	**1**	**2**	**3**	**4**	**5**	**6**	**7**
Insertion Probability (p)	0.01	0.05	0.10	0.40	1.00	1.00	1.00	1.00
Deletion Probability (q)	1.00	1.00	1.00	1.00	1.00	0.60	0.40	0.20

Figure 8: State transition logic for Proba and the corresponding mapping from states to insertion and deletion probabilities.

may introduce incorrect prefetches. Proba addresses this through an accuracy-coverage trade-off based on *marginal accuracy*. During PHT updates, trigger features are ranked by prediction accuracy. The most accurate trigger feature is updated using its absolute accuracy, while less accurate trigger features are evaluated only on the additional coverage they provide beyond more accurate trigger features. Specifically, for two trigger features A and B, where B is less accurate, we compute the marginal accurate-prefetch and marginal prediction sets as $C_{macc}[B] = C_{acc}[B] \wedge (\neg C_{pred}[A])$ and $C_{mpred}[B] = C_{pred}[B] \wedge (\neg C_{pred}[A])$, and use their ratio to determine whether to increase or decrease B's aggressiveness. If the accuracy of prefetches issued by B but not by A, considering only accesses not already covered by A, is above the accuracy target (80% by default), we increase B's aggressiveness; otherwise, we decrease it. This formulation extends to additional trigger features by excluding predictions already covered by more accurate trigger features. As a result, each trigger feature is rewarded only for the unique, correct prefetches it contributes, allowing Proba to improve coverage while maintaining high accuracy.

3.5 Jail Table and Region Sampling

Experimentally, we discovered that prior SMS-based prefetchers all suffer from imprecise Start-of-Generation (SoG) and End-of-Generation (EoG) tracking due to the finite capacity of the AGT. When deployed at a large cache level (e.g., L2), only a small fraction of entries reach EoG, where a block from the corresponding region is evicted from the cache (Figure 9). Most entries are instead evicted prematurely from the AGT, regardless of whether the prefetcher uses a filter table [36] to filter single-access regions. This causes the AGT to misinterpret later accesses to the same region as new trigger accesses. As a result, what should be a single generation is fragmented into several disjoint segments, and no individual update reflects the full sequence of accesses from the true SoG to the EoG event. This fragmentation reduces coverage in the PHT and corrupts accuracy estimation because useful prefetches that would have been accurate are misclassified, and it also leads to prefetches being issued at incorrect trigger accesses, causing the prefetcher to have misplaced aggression.

To address this, we divide the functionality of the AGT into two structures: one that tracks the interval between SoG and EoG, and another that records the access pattern observed within that interval. The interval-tracking requires only minimal storage, while the pattern-tracking does not need to capture every region because access patterns are ultimately aggregated in the PHT. This allows us to reduce the required capacity through statistical sampling.

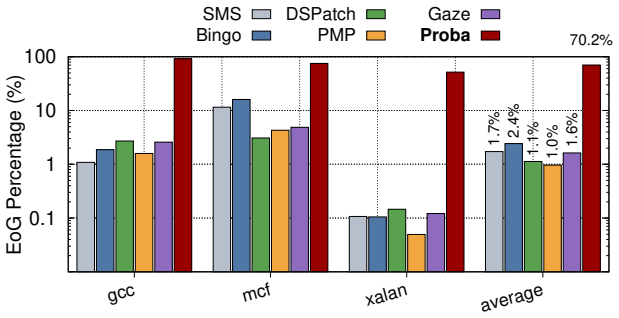


Figure 9: Percentage of Active Generation Table (AGT) entries that reach End-of-Generation (EoG) across SMS-based prefetchers on SPECspeed 2017.

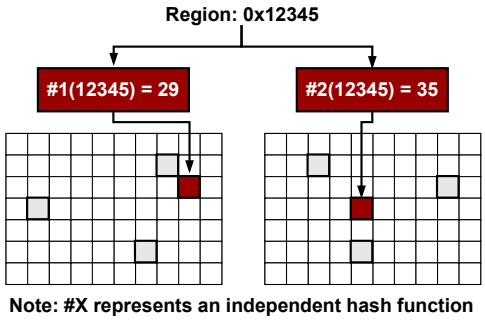


Figure 10: The Jail Table (JT) consists of two bitmaps indexed by different hash functions. When a region r is inserted, bits $\#1(r)$ and $\#2(r)$ are set in bitmap 1 and bitmap 2 respectively; both bits are cleared on EoG. A region is considered live in the cache only if both bits are set, in which case no prefetches are generated and no AGT entry is allocated.

For SoG-to-EoG tracking, we introduce a new structure called the Jail Table (JT). The JT (Figure 10) stores less information than the AGT[4], allowing it to track more regions with a small storage budget. The JT achieves this by only tracking whether a region is currently active (between SoG and EoG) in the cache, rather than recording its fine-grained access pattern. Upon a premature AGT eviction, the corresponding region is marked in the JT to ensure that subsequent accesses to this region are not reinserted into the AGT. The region is unmarked in the JT upon an EoG event (i.e., when any cache line of the region is evicted from the cache). When

[4]The percentage of entries that reach EoG rises from 2.7% with a 64-entry AGT but no JT to 6.1% with 128 entries and 53.5% with 512 entries. Proba reaches 70.2% using a 64-entry AGT with the JT. Increasing AGT capacity is less storage-efficient because it replicates pattern-tracking state solely to preserve SoG-to-EoG intervals.

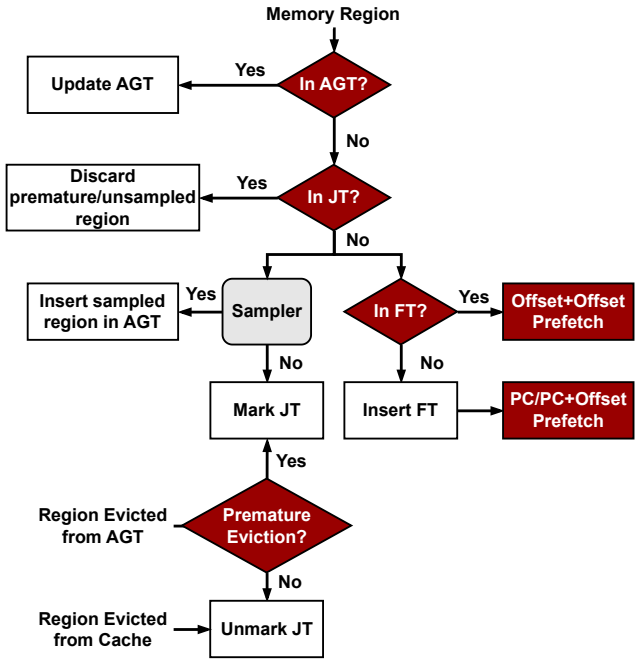


Figure 11: Sampling logic in Active Generation Table (AGT) and its interaction with the Jail Table (JT) and First-Offset Table (FT). If a region is already present in the AGT, all subsequent accesses update its bit pattern. If the region is not in the AGT but is marked in the JT (both bitmap bits set), the observation is ignored and no AGT entry or prefetch is generated until a later EoG event clears the JT entry. Otherwise, the region is potentially sampled and inserted into the AGT; regions not sampled are marked in the JT to prevent further AGT insertions. Any premature AGT eviction (before the region reaches its EoG) also marks its JT entry.

a region is marked in the JT, no prefetches are generated from it and no AGT entry is allocated to record its access pattern.

To track this, we use a pair of bitmaps inspired by a Bloom filter [8][5]. Two bitmaps are indexed by separate hash functions, #1 and #2. When a region number r is inserted (at SoG), bit #1(r) is set in bitmap 1 and #2(r) is set in bitmap 2. Deletion (at EoG) clears these two bits. To test whether a region is within the SoG-EoG interval, we check whether bitmap 1 contains bit #1(r) and bitmap 2 contains bit #2(r).

To increase the number of regions that reach EoG and thereby generate frequent PHT updates, the AGT switches to *sampling* under pressure (when a set reaches 50% occupancy): new regions are inserted into the AGT with a sampling rate of 5% (Figure 11). To prevent accesses to unsampled regions from entering the AGT, we reuse the JT: such regions are marked in the JT at first access, rather than only upon premature AGT evictions. If a region is already marked in the JT, it is not allocated to the AGT again until its EoG event is observed. Unlike SMS and DSPatch, Proba employs a Random Replacement Policy (RandomRP) in the AGT to support unbiased sampling. This sampling significantly reduces AGT traffic while providing a sufficiently broad set of observations for high-quality PHT updates.

The PHTs are updated when either an EoG event occurs in the cache or the AGT prematurely evicts an entry, but due to sampling, these premature evictions are now much rarer and so comparatively more of the AGT entries reach the intended EoG point (Figure 9).

[5]The Jail Table differs from a Bloom filter, which uses a single bit array indexed by multiple hashes and does not support deletion without clearing or augmenting the structure, and from Bloom-like structures such as Cuckoo filters [14], which organize state differently. The Jail Table supports efficient deletion and deliberately permits both false positives and false negatives. Because deleting an entry removes the corresponding element from each array, it may also remove an element associated with another still-live region, thereby introducing a false negative. False negatives merely cause region splitting, whereas false positives reduce prefetch coverage.

Table 1: Detailed storage requirements of Proba

Structure	Description	Storage
FT	8-way; 64-entry, LRURP • Region Tag (36b) • Trigger Offset (6b) • LRU State (3b)	360B
AGT	8-way; 64-entry, RandomRP • Region Tag (36b) • Trigger PC (16b) • Trigger Offset (6b) • Second Offset (6b) • Observation Bit Pattern (64b)	1024B
JT	Bloom-like filter, dual hash • Array (2×4096b)	1024B
PHT	16-way, 256-entry, BRRIPRP • Tag (12b) • Proba Bit Pattern (64b) • State (3b) • BRRIP State (3b)	3×2624B
Total		**10.04KiB**

3.6 Hardware Overhead

Overall, Proba requires 10.04 KiB of storage (Table 1). Proba tracks 4 KiB regions, which correspond to the size of a typical physical page. With 64-byte cache lines, each region contains $4096/64 = 64$ cache-line offsets, so each spatial pattern is represented by a 64-bit pattern. Each region entry also stores a 36-bit region tag, a 6-bit trigger offset, and, where required, a 6-bit second offset. The First-Offset Table (FT) stores only the region tag and trigger offset, since it is used only to trigger the `Offset+Offset` trigger feature. The Active Generation Table (AGT) stores both the trigger and second offsets so that the `Offset+Offset` trigger feature can be trained when an entry is evicted. Both FT and AGT are 64-entry, 8-way set-associative structures; FT uses LRU replacement, while AGT uses Random Replacement Policy (RandomRP). Pattern History Tables (PHTs) support three trigger features: `PC-Only`, `PC+Offset`, and `Offset+Offset`. Each PHT is a 256-entry, 16-way set-associative structure using BRRIP replacement. The Jail Table (JT) uses two hash functions and two 4096-bit bitmaps.

Table 2: Simulation parameters for baseline system

Core	Out-of-order, TAGE-SC-L Branch Pred., 4 GHz, 8-issue width, 8-retire width, 576-entry ROB
TLBs	L1 iTLB & dTLB: 64 entries, 4-way, 1 cycle STLB: 1536 entries, 12-way, 8 cycles
L1I	64 KiB, 8-way, 4 cycles
L1D	48 KiB, 12-way, 5 cycles, LRURP
L2C	2 MiB, 16-way, 10 cycles, LRURP
LLC	3 MiB / core, 12-way, 35 cycles, DRRIPRP
MSHRs	8/16/32/64 in L1I/L1D/L2/LLC
PQs	32/8/16/32 in L1I/L1D/L2/LLC
DRAM	128GB, 2400 MTPS (4800 multicore), 8 banks

Table 3: Parameters of each comparison prefetcher

Name	Parameters	Storage
SMS [36]	4KiB region, 64-entry AGT, 1024-entry PHT, **Index: `PC+Offset`**	11.95KiB
Bingo [3]	4KiB region, 64-entry AGT, 1024-entry PHT, **Index: `PC+Offset/PC+Addr`**	13.20KiB
DSPatch [6]	4KiB region, 64-entry Page Buf., 1024-entry SPT, **Index: `PC-Only`**	9.68KiB
PMP [21]	4KiB region, 64-entry AGT, 64-entry OPT, 256-entry PPT, **Index: `Offset/PC-Only`**	9.02KiB
Gaze [11]	4KiB region, 64-entry AGT, 1024-entry PHT, 8-entry DPCT, **Index: `Offset+Offset`**	10.81KiB

4 Experimental Setup

We evaluate Proba with the trace-based ChampSim simulator [17], using the configuration in Table 2. We compare Proba with several state-of-the-art SMS-based prefetchers, with detailed configurations reported in Table 3. For fairness, all evaluated prefetchers are placed at the L2 cache, track 4 KiB spatial regions, and are configured to use a comparable storage budget. All configurations, including the baseline, include a degree-3 IP-stride prefetcher [10] at the L1 cache. Prefetcher implementations are taken from the original authors [3, 6, 11, 21, 36].

The single-core configuration uses one DRAM channel at 2400 MTPS. For multicore experiments, we use a data rate of 4800 MTPS and two DRAM channels for both the 2-core and 4-core configurations; the 4-core configuration additionally uses two ranks per channel. This configuration is inspired by the one used in Gaze [11].

We evaluate on simpoint traces from four benchmark suites: SPECspeed 2017 with 95 traces, SPEC CPU2006 with 93 traces, Ligra with 76 traces, and CloudSuite with 120 traces. For each trace, we use a 200-million-instruction warmup followed by a 200-million-instruction simulation window. Results are aggregated across simpoints of each workload, with weights where available.

5 Evaluation

5.1 Single Core Analysis on SPECspeed 2017

In the breakdown of SPECspeed 2017 (Figure 12), we show that Proba delivers a 6.6% geomean speedup while incurring only 8.5% additional DRAM traffic. PMP [21] is the next-best design in terms of performance, at 5.6% speedup, but does so with a substantially higher 28.1% DRAM traffic overhead. This is despite PMP using large 5-bit saturating counters to track patterns across historical updates, whereas Proba uses just a single probabilistic bit to do the same. Indeed, PMP's high traffic causes slowdowns in `omnet` and `cam4`, where Proba instead behaves in an accuracy-seeking manner. PMP also slightly slows `mcf` by 0.8%, whereas Proba improves its performance by 3.3%, while generating substantially less traffic. In our setup, Gaze [11] is less performant than PMP, achieving a 5.0% speedup. Although its traffic is considerably more modest than PMP's, at 12.4% above baseline, it is still less frugal than Proba. We attribute this to Gaze's reliance on a single `Offset+Offset` trigger feature, which reduces coverage in workloads such as `gcc` and `mcf`. While Gaze includes an integrated stride prefetcher, its benefit is limited due to overlap with the L1 stride prefetcher in the baseline. In some cases, Gaze outperforms Proba by being more aggressive, but more often the resulting increase in DRAM traffic leads to lower overall performance, even in this single-threaded setting with ample bandwidth. DSPatch, Bingo, and SMS also underperform Proba in geomean speedup while generating substantially more DRAM traffic. A particularly strong per-workload exception is `cactuBSSN`, where DSPatch's pattern-compression mechanism (Section 2.2) happens to increase coverage with little accuracy loss. This benefit is specific to compression rather than DSPatch's underlying PHT update policy, and disappears when DSPatch is evaluated without compression. As previously shown in Figure 1, the performance gap grows when DRAM bandwidth is reduced relative to default. As we shall see, this is also reflected when effective bandwidth is reduced by contention in multicore workloads.

Beyond these performance and traffic trends, we can also consider prefetcher behavior over phase transitions. For example, under Proba, the IPC over 10-million-instruction intervals for the `wrf_s-6673B` trace drops sharply from 3.62 in one interval to 0.90 in the next, while the `pop2-17B` trace drops from 2.25 in one interval to 1.57 in the next. Despite these abrupt changes in workload behavior, Proba achieves the highest speedup among the evaluated prefetchers on both workloads, suggesting that its probabilistic adaptation remains effective across sharp phase transitions.

5.2 Alternative Benchmark Suites

In other suites, Proba's DRAM traffic reduction translates into larger performance gains. Figure 13 shows that, on SPEC CPU2006, Proba outperforms the closest competitor, Gaze, by 1.7% while incurring roughly one third of its additional DRAM traffic, and outperforms PMP by 2.6%. Proba also achieves large gains on workloads such as `mcf`, where it improves performance by 29.7%, compared with 26.0% for DSPatch and substantially smaller speedups for the other prefetchers. This suggests that Proba's multiple trigger features help distinguish useful spatial patterns that other designs fail to capture accurately. Ligra also benefits substantially from Proba, largely due to its very low traffic overhead of only 1.7%. The advantage

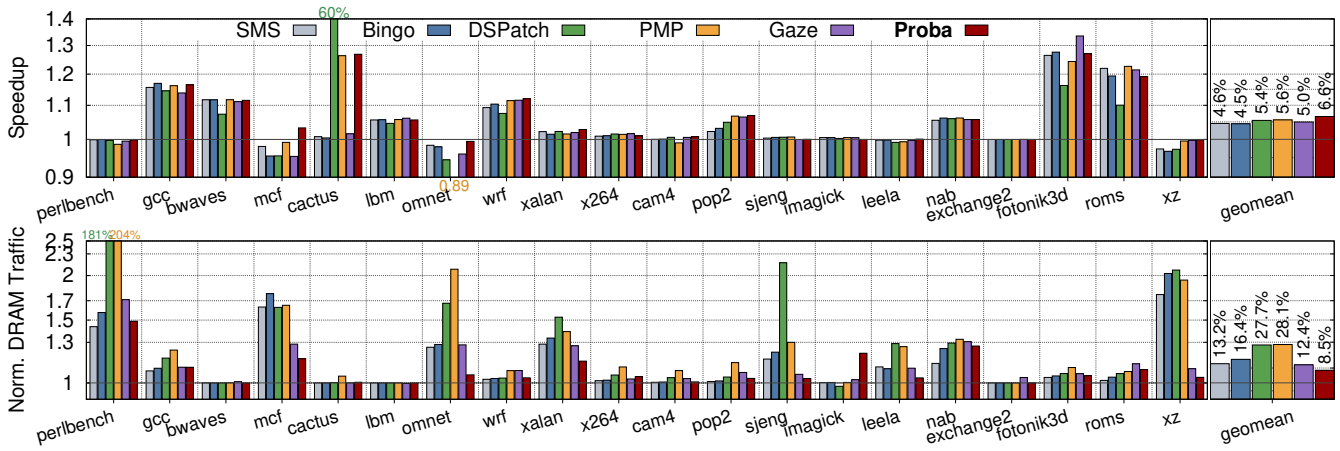


Figure 12: Single-core performance and normalized DRAM traffic of the evaluated SMS-based prefetchers on SPECspeed 2017.

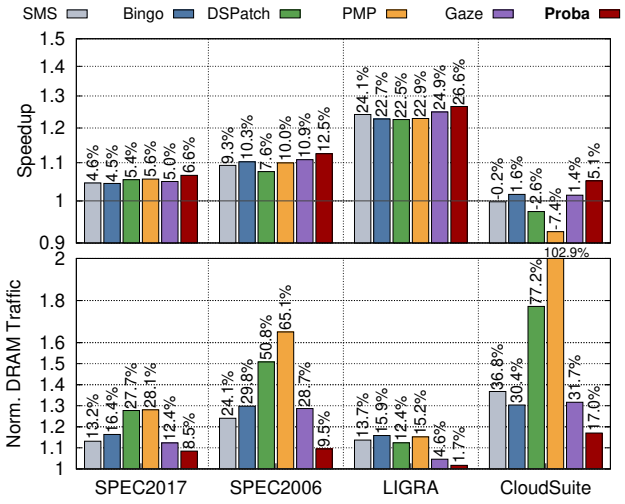


Figure 13: Single-core speedup and normalized DRAM traffic of prefetchers on various benchmark suites (geomean).

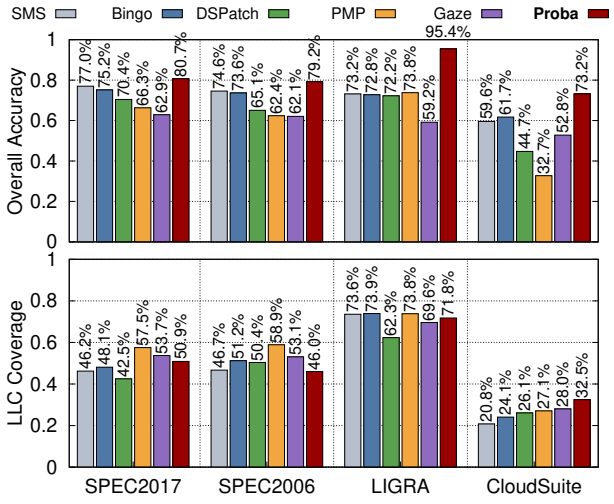


Figure 14: Overall Accuracy and LLC Coverage of prefetchers on various benchmark suites (average).

is even more pronounced on CloudSuite: Proba achieves a 5.1% speedup above baseline, outperforming every competitor across all workloads. In these data-center workloads, for which SMS was originally designed [36], Proba is not only more accurate but also achieves higher coverage than other SMS-based prefetchers.

Indeed, Figure 14 shows that Proba consistently achieves the highest overall prefetch accuracy[6], as expected from its low DRAM traffic. LLC coverage[7] is sometimes lower than that of competing designs, but this is intentional: when a workload is poorly suited to SMS and accurate predictions are difficult, Proba reduces coverage to avoid harmful DRAM traffic, preserving performance rather than pursuing aggressive but low-value prefetches.

[6]Overall prefetch accuracy is calculated as (L2 useful prefetches + LLC useful prefetches) / (L2 useful prefetches + LLC useful prefetches + L2 useless prefetches + LLC useless prefetches). This accounts for prefetchers such as PMP and Gaze, which can prefetch into both the L2 and LLC.

[7]LLC coverage is calculated as 1 - (LLC demand misses with prefetching / LLC demand misses without prefetching).

5.3 Multicore Performance Analysis

Speedups from Proba's reduced traffic become even more prominent in multicore settings, where the shared DRAM becomes a critical bottleneck. Figure 15 shows a breakdown of the workloads in CloudSuite, where Proba achieves a geomean speedup of 7.9% for 4-core execution, almost double that of the nearest contender. DSPatch and PMP suffer significant slowdowns in `classification` and `cloud9`, as their higher traffic becomes harmful under contention. While Proba also sees slight slowdowns in these workloads, they are outweighed by substantial gains in `nutch` and `streaming`.

Figure 16 shows homogeneous mixes of the same memory-intensive SPEC workloads (`mcf`, `gcc`, `lbm`, `omnet` and `xalan`, all from both 2006 and 2017) running on 2 and 4 cores. While Proba's advantage over PMP is moderate in the single-core setting (Figure 12), it becomes much larger under multicore execution (5.7% vs. -0.3% for 4-core, and 7.0% vs. 1.1% for 2-core). This is because the additional DRAM traffic from most prefetchers becomes increasingly detrimental as cores scale. Among all designs, only Gaze and Proba maintain reasonable traffic levels; however, Gaze still

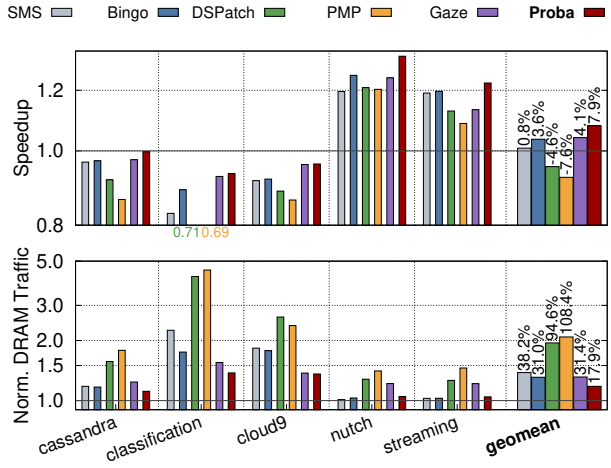


Figure 15: Speedup and DRAM traffic for 4-core CloudSuite.

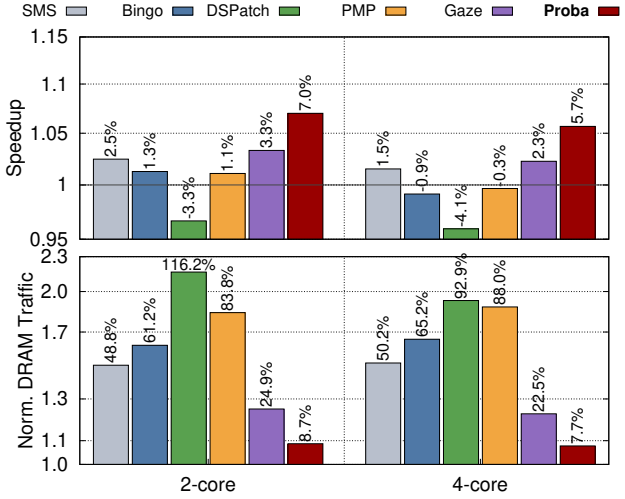


Figure 16: Geomean speedup and normalized DRAM traffic for 2-core and 4-core homogeneous mixes of memory-intensive SPECspeed 2017 and SPEC CPU2006 workloads.

underperforms (2.3% for 4-core and 3.3% for 2-core) due to lower coverage in performance-critical workloads.

5.4 Ablation Study

Figure 17 shows how components of Proba contribute to the performance and DRAM traffic, starting from an SMS baseline and progressively adding each component. Training on L2 misses and prefetch tag hits (Section 3.1) improves DRAM efficiency and is essential for Proba's accuracy targeting, as it prevents redundant cache hits from being misclassified as accurate prefetches. The dynamic probabilistic update mechanism (Section 3.2 and Section 3.3), which is the core of Proba's design, is crucial for reducing DRAM traffic while maintaining competitive speedup. Introducing multi-feature Pattern History Tables with `PC-Only` and `Offset+Offset` trigger features improves coverage by capturing a wider range of spatial correlations. Adding marginal accuracy decision-making (Section 3.4) further improves the balance between speedup and

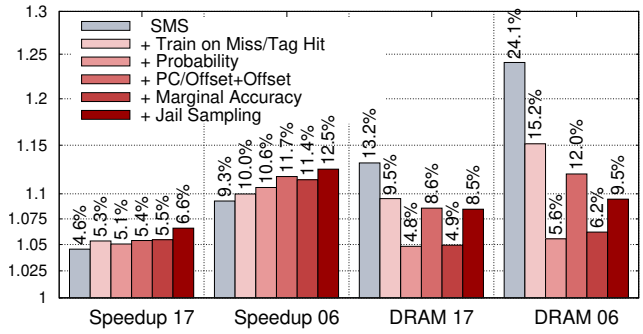


Figure 17: Ablation study. Speedup and DRAM traffic shown for SPECspeed 2017 and SPEC CPU2006 (geomean).

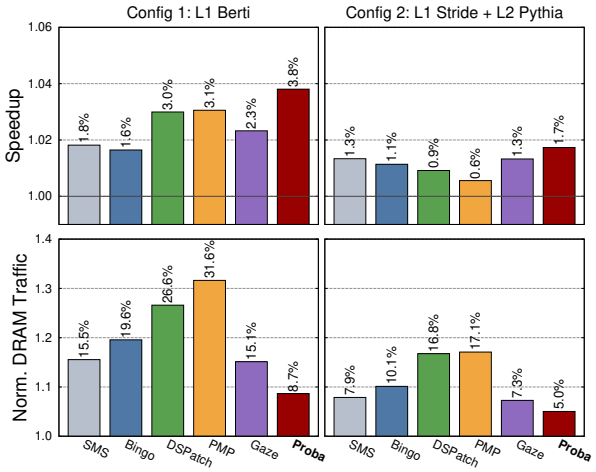


Figure 18: Geomean speedup and normalized DRAM traffic on SPECspeed 2017 for two multi-prefetcher configurations: Config 1 uses Berti at L1 with an SMS-based prefetcher at L2, while Config 2 uses a degree-3 stride prefetcher at L1 and Pythia together with an SMS-based prefetcher at L2.

traffic by filtering predictions based on their expected benefit. Finally, the Jail Table and region sampling (Section 3.5) provide more complete observations, improve Proba's accuracy estimates, and help bring prefetch aggressiveness closer to the desired level, boosting speedup while moderately increasing DRAM traffic.

5.5 Integration with Multi-Prefetcher Setups

Proba may be used as the spatial component of multi-prefetcher stacks, such as those in Neoverse V2 [9], complementing prefetchers for other access patterns. Figure 18 evaluates two such configurations. In Config 1, Berti [30] is placed at L1 with each SMS-based prefetcher at L2. On SPECspeed 2017, Proba achieves a 3.8% geomean speedup versus 3.1% for the next-best design, PMP, while adding only 8.7% DRAM traffic, compared with 15.1% for Gaze, the next most traffic-efficient alternative. Config 2 uses a stride prefetcher at L1 and Pythia [5] alongside each SMS-based prefetcher at L2. Proba improves performance by 1.7% over the corresponding non-SMS stack, versus 1.3% for SMS and Gaze, while again incurring the lowest DRAM traffic overhead of 5.0%. Thus, even as the

underlying prefetcher stack strengthens, Proba provides the best performance-traffic trade-off.

5.6 Parameter Sensitivity

In general, Proba is remarkably insensitive to parameters and random variance, because its design is based on broader principles involving stochastic processes rather than tuning.

Jail Table Size: Using a substantially larger JT has negligible impact on performance, implying very few false positives and negatives. Across SPECspeed 2017, increasing each JT bitmap from 4096 to 16384 bits improves geomean speedup only from 6.6% to 6.7%. Reducing each JT bitmap to 1024 bits increases collision-induced false positives/negatives for some workloads, causing region splitting and less accurate generation tracking. Speedup decreases from 26.8% to 23.7% for `cactuBSSN`, and from 19.1% to 17.7% for `roms`.

AGT Sampling Rate: We swept the AGT sampling rate from 1% to 50% on SPECspeed 2017. At a 50% sampling rate, only 7.5% of regions reach EoG. Reducing the sampling rate to 1% increases the EoG percentage to 90.8%, but also increases DRAM traffic overhead to 10.2% because fewer regions are captured by the AGT. Compared with the 5% default, 50% reduces speedup by 0.2 percentage points, while 1% achieves similar speedup.

Accuracy Threshold: Proba is surprisingly insensitive to the accuracy threshold (*AccThr*) over a wide range of settings. In SPECspeed 2017, the geomean speedup varies only slightly, from 6.59% at *AccThr*=80 to 6.66%, 6.66%, and 6.59% at *AccThr*=40, 60, and 20, respectively. However, a higher threshold is beneficial for memory-intensive workloads because it suppresses unnecessary traffic. For example, with *AccThr*=80, normalized DRAM traffic overhead for `mcf` falls to 16.9%, compared with 27.0%, 25.3%, and 23.7% at *AccThr*=20, 40, and 60, respectively. We attribute this insensitivity to marginal accuracy (Section 3.4) when measured over multiple PHTs, which causes filtering out of low-quality patterns across a broad range of thresholds: we observed that single-PHT variants of Proba were more sensitive to *AccThr*.

Number of States per PHT Entry: We also varied the number of transition state bits (Figure 8) from 1 to 4, corresponding to 2 to 16 probability states. This also has negligible performance impact across SPECspeed 2017: 1-bit, 2-bit, 3-bit and 4-bit Proba all give 6.6% geomean speedup, though 1-bit and 4-bit increase DRAM traffic overhead to 9.9% and 9.0% respectively. Proba is largely insensitive to the granularity of the probability as long as it can move between several states.

Larger PHTs: Proba is not sensitive to PHT sizes within reasonable ranges. Halving the number of entries per PHT to 128 reduces SPECspeed 2017's speedup from 6.59% to 6.57%, and increasing to 512 increases speedup to 6.64%. DRAM traffic is only marginally more sensitive, reducing to 8.1% overhead at 128 entries (compared with 8.45% default) and increasing to 8.71% with 512 entries.

6 Related Work

Prefetching has long been an extensively studied approach to tolerate high memory latency, with a wide range of algorithms [13, 25]. We discuss the most relevant below.

Spatial Memory Streaming Prefetchers: Spatial Memory Streaming (SMS) [36] extracts spatial correlations across memory regions using bit patterns indexed by the `PC+Offset` trigger feature. Extensions of SMS [15] use rotated bit patterns to remove the offset from the trigger feature, reducing storage requirements. Later work on SMS [35] provides three static variants: SMS-AND (favoring accuracy), SMS-OR (favoring coverage), and Saturating Counters (balancing accuracy and coverage). SMS-AND and SMS-OR store two histories and apply bitwise AND or OR to issue prefetches. More recently, Bingo [3] extended SMS by leveraging short- and long-term histories, inspired by the TAGE branch predictor [33]. Unlike SMS, which relies on one type of trigger feature, Bingo unifies `PC+Offset` and `PC+Address` trigger features in one prediction table. DSPatch [6] improves upon SMS by trading off accuracy and coverage at runtime. It stores two predictions: one biased toward accuracy and one toward coverage. DSPatch dynamically selects between them based on bandwidth utilization. Gaze [11] uses the temporal `Offset+Offset` trigger feature to index its prediction tables. PARS [23] supports multiple region sizes. PMP [21] uses saturating-counter updates and maintains two PHT tables: one based on `PC-Only` and one on `Offset-Only`.

Aggression Control: Feedback-directed [37] prefetching trades off accuracy and coverage by estimating prefetch accuracy, prefetch timeliness, and prefetch-induced cache pollution to adjust prefetch aggressiveness. More recent work [7] uses Perceptron-based Prefetch Filtering (PPF) to allow the underlying prefetcher to achieve high coverage while filtering out inaccurate prefetches. Similar to our work, DSPatch [6] uses table-based accuracy estimates to guide accuracy-coverage seeking decisions and control aggression.

Other classes of prefetchers: Beyond spatial memory streaming, other classes of prefetchers exist, some of which are often used alongside SMS-based prefetchers. Stride prefetchers track linear patterns with low complexity but limited coverage [10]. More advanced schemes such as Best-Offset Prefetching (BOP) [24] and Berti [30] track more generalized past offsets. Signature Path Prefetcher (SPP) [22] captures correlation across multiple load PCs. Address-delta correlation can also be implemented in global history buffers [31]. Neural-network-based prefetchers [12, 34] attempt to generalize complex patterns, though at higher cost. Temporal prefetchers [2, 38–40] exploit recurrence in access sequences, while runahead schemes [19, 26–29, 32] explore transient pre-execution.

7 Conclusion

This paper has presented Proba, a probabilistic update mechanism for SMS-based prefetchers. By adaptively updating each spatial bit according to accuracy-aware probabilities, Proba preserves long-term access behavior while avoiding pattern corruption, duplicated storage, and coarse-grained update policies in prior designs such as DSPatch and PMP. It further incorporates heterogeneous trigger features from recent SMS-based prefetchers, including Gaze, while controlling their aggressiveness through marginal accuracy. Across our evaluation, Proba consistently outperforms state-of-the-art SMS-based prefetchers, achieving higher speedups with substantially lower DRAM traffic. These results show that probabilistic pattern updates, combined with marginal-accuracy-based aggression control and multi-feature PHTs, provide an effective and practical direction for spatial prefetching in next-generation processors.

8 Artifact Appendix

8.1 Abstract

This artifact contains the source code and simulation framework used to evaluate Proba. The repository contains:

- Source code for Proba.
- Source code for SMS [36], Bingo [4], DSPatch [6], PMP [21], and Gaze [11].
- The modified ChampSim framework and system configuration used to evaluate all presented prefetchers.
- Scripts to download SPECspeed 2017 traces, run simulations, and collect results.

This artifact reproduces the SPECspeed 2017 performance, DRAM traffic, coverage, and accuracy results presented in the paper. In particular, it reproduces the data reported in Figures 12–14.

8.2 Artifact check-list (meta-information)

- **Program:** ChampSim
- **Data set:** SPECspeed 2017, downloaded using the provided scripts
- **Metrics:** Speedup, normalized DRAM traffic, LLC coverage, and overall prefetch accuracy
- **Output:** Tables containing per-benchmark and aggregate metrics for each prefetcher
- **Experiments:** Baseline and Proba. Optional: SMS, Bingo, DSPatch, PMP, and Gaze
- **How much disk space is required?** Approximately 40 GB
- **How much time is needed to prepare the workflow?** Approximately 1 hour to download the workload traces
- **How much time is needed to complete the experiments?** Approximately 6 hours to reproduce the main results (baseline and Proba). Optionally, approximately 21 hours are required to reproduce the results for the baseline and all evaluated prefetchers. These times were measured on an Intel Xeon Platinum 8168 machine using 40 cores.
- **Publicly available?** Yes
- **Code license:** Apache License 2.0
- **Archived (provide DOI):** https://doi.org/10.5281/zenodo.21859952

8.3 Description

8.3.1 How to access. The source code is publicly available on GitHub at https://github.com/justinhuang666666/Proba_Artifact and on Zenodo at https://doi.org/10.5281/zenodo.21859952.

8.3.2 Software dependencies. Installing and running this artifact requires the following software dependencies:

- Python 3.9 or later
- GCC/G++ 11
- GNU Make
- Git

The artifact was tested with the following system configuration:

- Ubuntu 22.04.5 LTS
- Linux kernel 5.15.0
- GCC/G++ 11.4.0
- Python 3.10.12

GCC/G++ 11 is required to compile the artifact. Newer compiler versions are not currently supported. The required compiler and other dependencies can be installed automatically using the setup script.

8.3.3 Data sets. This artifact contains a script to download the SPECspeed 2017 traces used in DPC3 [1], hosted at https://dpc3.compas.cs.stonybrook.edu/champsim-traces/speccpu/.

By default, the downloaded SPECspeed 2017 traces are stored in:

```
traces/spec2017/
```

This artifact can also be modified to reproduce the results for the other evaluated benchmark suites. The SPEC CPU2006 traces are hosted in the same archive as the SPECspeed 2017 traces. The Ligra traces are available at https://zenodo.org/records/14267978, while the CloudSuite traces are available through the Second Cache Replacement Championship (CRC-2) at https://crc2.ece.tamu.edu/.

8.4 Installation

Use the following commands to clone the repository, build the ChampSim dependencies, and download the required traces:

```
# Clone the repository and initialize the artifact:
git clone
cd Proba_Artifact
./scripts/setup.sh

# Download the SPECspeed 2017 traces
python3 scripts/download_spec2017_traces.py
```

8.5 Experiment workflow

In the commands below, replace `N` with the number of simulations to execute in parallel. For example, use `-j 40` to execute up to 40 simulations concurrently.

The evaluation baseline contains a degree-3 stride prefetcher at L1D and no prefetcher at L2C. Therefore, the `no` configuration denotes no *L2C* prefetching, rather than no prefetching anywhere in the cache hierarchy.

8.5.1 Reproducing Proba.

```
# Run the baseline and Proba configurations
python3 scripts/run_experiments.py no -j N
python3 scripts/run_experiments.py proba -j N
```

Each configuration takes approximately 3 hours on an Intel Xeon Platinum 8168 system when running 40 simulations in parallel. Systems with fewer available cores should use a smaller value of `N`, with a corresponding increase in total execution time.

Each simulation produces a text file containing the simulator log and a JSON file containing the statistics used by the result-reporting script:

```
results/spec2017/<prefetcher>/<benchmark>/<trace>.txt
results/spec2017/<prefetcher>/<benchmark>/<trace>.json
```

8.5.2 Optional: reproducing all evaluated prefetchers.

```
# Run the baseline and all evaluated prefetchers
for pref in no sms bingo dspatch pmp gaze proba; do
    python3 scripts/run_experiments.py "\$pref" -j N
done
```

With `N=40`, the complete experiment takes approximately 21 hours on an Intel Xeon Platinum 8168 system.

8.6 Evaluation and expected results

The `scripts/print_results.py` script produces tables containing the per-benchmark and aggregate results for:

- IPC speedup over the baseline;
- normalized DRAM traffic;
- LLC prefetch coverage; and
- overall prefetch accuracy across both L2C and LLC.

Accuracy is measured across both cache levels to ensure a fair comparison with prefetchers that employ multilevel prefetching.

8.6.1 Reproducing Proba. After both the baseline and Proba simulations have completed, run:

```
# Print the Proba results
python3 scripts/print_results.py proba
```

The aggregate results should be:

```
=====================================================
> Overall summary
=====================================================
Prefetcher    Speedup   DRAM      LLC cov.  Accuracy
----------    --------  --------  --------  --------
proba         1.0659    1.0845    0.5088    0.8068
```

8.6.2 Optional: reproducing all evaluated prefetchers. After all simulations have completed, run:

```
# Print all prefetchers results
python3 scripts/print_results.py sms bingo dspatch pmp
     gaze proba
```

The aggregate results should be:

```
=====================================================
> Overall summary
=====================================================
Prefetcher    Speedup   DRAM      LLC cov.  Accuracy
----------    --------  --------  --------  --------
sms           1.0456    1.1316    0.4622    0.7699
bingo         1.0446    1.1638    0.4808    0.7516
dspatch       1.0544    1.2772    0.4247    0.7042
pmp           1.0559    1.2806    0.5751    0.6629
gaze          1.0497    1.1236    0.5372    0.6285
proba         1.0659    1.0845    0.5088    0.8068
```

8.7 Experiment customization

Proba is implemented in `prefetcher/proba/`. You can change the parameter of Proba in `prefetcher/proba/proba.h`. The simulated processor and cache parameters can be changed in `params/baseline.json`. The experiment scripts under `scripts/` can be modified to evaluate additional workloads, configurations, and prefetchers.

8.8 Methodology

Submission, reviewing, and badging methodology:

- https://www.acm.org/publications/policies/artifact-review-and-badging-current
- https://cTuning.org/ae

References

[1] Champsim traces for spec cpu. https://dpc3.compas.cs.stonybrook.edu/champsim-traces/speccpu/, 2025. Accessed: 2025-11-15.

[2] Ainsworth, S., and Mukhanov, L. Triangel: A high-performance, accurate, timely on-chip temporal prefetcher. In *51st ACM/IEEE Annual International Symposium on Computer Architecture* (2024), ISCA '24.

[3] Bakhshalipour, M., Shakerinava, M., Lotfi-Kamran, P., and Sarbazi-Azad, H. Bingo spatial data prefetcher. In *2019 IEEE International Symposium on High Performance Computer Architecture* (2019), HPCA '19.

[4] Bakhshalipour, M., Shakerinava, M., Lotfi-Kamran, P., and Sarbazi-Azad, H. Bingo: Spatial Data Prefetcher Repository. https://github.com/bakhshalipour/Bingo, 2019. GitHub repository.

[5] Bera, R., Kanellopoulos, K., Nori, A., Shahroodi, T., Subramoney, S., and Mutlu, O. Pythia: A customizable hardware prefetching framework using online reinforcement learning. In *54th Annual IEEE/ACM International Symposium on Microarchitecture* (2021), MICRO-54.

[6] Bera, R., Nori, A. V., Mutlu, O., and Subramoney, S. DSPatch: Dual spatial pattern prefetcher. In *52nd Annual IEEE/ACM International Symposium on Microarchitecture* (2019), MICRO-52.

[7] Bhatia, E., Chacon, G., Pugsley, S., Teran, E., Gratz, P. V., and Jiménez, D. A. Perceptron-based prefetch filtering. In *46th ACM/IEEE Annual International Symposium on Computer Architecture* (2019), ISCA '19.

[8] Bloom, B. H. Space/time trade-offs in hash coding with allowable errors. *Commun. ACM 13*, 7 (July 1970), 422–426.

[9] Bruce, M. Arm Neoverse V2 platform: Leadership Performance and Power Efficiency for Next- Generation Cloud Computing, ML and HPC Workloads. https://hc2023.hotchips.org/assets/program/conference/day1/CPU1/HC2023.Arm.MagnusBruce.v04.FINAL.pdf.

[10] Chen, T.-F., and Baer, J.-L. Effective hardware-based data prefetching for high-performance processors. *IEEE Transactions on Computers 44*, 5 (1995).

[11] Chen, Z., Wu, C., Gu, Y., Jia, R., Li, J., and Guo, M. Gaze into the pattern: Characterizing spatial patterns with internal temporal correlations for hardware prefetching. In *2025 IEEE International Symposium on High Performance Computer Architecture* (2025), HPCA '25.

[12] Dong, Q., Jain, A., and Lim, C. A new formulation of neural data prefetching. In *51st Annual International Symposium on Computer Architecture* (2024), ISCA '24.

[13] Falsafi, B., and Wenisch, T. F. *A Primer on Hardware Prefetching.* Synthesis Lectures on Computer Architecture. Springer Cham, Cham, Switzerland, 2014.

[14] Fan, B., Andersen, D. G., Kaminsky, M., and Mitzenmacher, M. D. Cuckoo filter: Practically better than bloom. In *10th ACM International on Conference on Emerging Networking Experiments and Technologies* (2014), CoNEXT '14.

[15] Ferdman, M., Somogyi, S., and Falsafi, B. Spatial memory streaming with rotated patterns. *Journal of Instruction-Level Parallelism (JILP) 13* (2011).

[16] Ghose, S., Yaglikçi, A. G., Gupta, R., Lee, D., Kudrolli, K., Liu, W. X., Hassan, H., Chang, K. K., Chatterjee, N., Agrawal, A., O'Connor, M., and Mutlu, O. What your dram power models are not telling you: Lessons from a detailed experimental study. *Proceedings of the ACM on Measurement and Analysis of Computing Systems (POMACS) 2*, 3 (Dec. 2018).

[17] Gober, N., Chacon, G., Wang, L., Gratz, P. V., Jimenez, D. A., Teran, E., Pugsley, S., and Kim, J. The championship simulator: Architectural simulation for education and competition. https://arxiv.org/abs/2210.14324, 2022.

[18] Guo, Y., Chheda, S., Koren, I., Krishna, M., and Moritz, C. Energy characterization of hardware-based data prefetching. In *IEEE International Conference on Computer Design: VLSI in Computers and Processors* (2004), ICCD 2004.

[19] Hashemi, M., Mutlu, O., and Patt, Y. N. Continuous runahead: Transparent hardware acceleration for memory intensive workloads. In *49th Annual IEEE/ACM International Symposium on Microarchitecture* (2016), MICRO-49.

[20] Jaleel, A., Theobald, K. B., Steely, S. C., and Emer, J. High performance cache replacement using re-reference interval prediction (RRIP). In *37th Annual International Symposium on Computer Architecture* (2010), ISCA '10.

[21] Jiang, S., Yang, Q., and Ci, Y. Merging similar patterns for hardware prefetching. In *55th Annual IEEE/ACM International Symposium on Microarchitecture* (2022), MICRO-55.

[22] Kim, J., Pugsley, S. H., Gratz, P. V., Reddy, A. N., Wilkerson, C., and Chishti, Z. Path confidence based lookahead prefetching. In *2016 49th Annual IEEE/ACM International Symposium on Microarchitecture* (2016), MICRO-49.

[23] Lin, Y., Lin, W., Xu, J., Chen, Y., Jin, Z., Qin, J., He, J., Cai, S., Zhang, Y., Wang, Z., and Chen, W. PARS: A pattern-aware spatial data prefetcher supporting multiple region sizes. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems 43*, 11 (2024), 3638–3649.

[24] Michaud, P. Best-offset hardware prefetching. In *2016 IEEE International Symposium on High Performance Computer Architecture* (2016), HPCA '16.

[25] Mittal, S. A survey of recent prefetching techniques for processor caches. *ACM Comput. Surv. 49*, 2 (aug 2016).

[26] Mutlu, O., Stark, J., Wilkerson, C., and Patt, Y. N. Runahead execution: An alternative to very large instruction windows for out-of-order processors. In

9th International Symposium on High-Performance Computer Architecture (2003), HPCA '03.

[27] Naithani, A., Ainsworth, S., Jones, T. M., and Eeckhout, L. Vector runahead. In *48th Annual International Symposium on Computer Architecture* (2021), ISCA '21.

[28] Naithani, A., Feliu, J., Adileh, A., and Eeckhout, L. Precise runahead execution. In *26th International Symposium on High-Performance Computer Architecture* (2020), HPCA '20.

[29] Naithani, A., Roelandts, J., Ainsworth, S., Jones, T. M., and Eeckhout, L. Decoupled vector runahead. In *56th Annual IEEE/ACM International Symposium on Microarchitecture* (2023), MICRO-56.

[30] Navarro-Torres, A., Panda, B., Alastruey-Benedé, J., Ibáñez, P., Viñals-Yúfera, V., and Ros, A. Berti: an accurate local-delta data prefetcher. In *55th IEEE/ACM International Symposium on Microarchitecture* (2022), MICRO-55.

[31] Nesbit, K., and Smith, J. Data cache prefetching using a global history buffer. In *10th International Symposium on High Performance Computer Architecture* (2004), HPCA '04.

[32] Roelandts, J., Naithani, A., Ainsworth, S., Jones, T. M., and Eeckhout, L. Scalar vector runahead. In *57th IEEE/ACM International Symposium on Microarchitecture* (2024), MICRO-57.

[33] Seznec, A. A new case for the tage branch predictor. In *44th Annual IEEE/ACM International Symposium on Microarchitecture* (2011), MICRO-44.

[34] Shi, Z., Jain, A., Swersky, K., Hashemi, M., Ranganathan, P., and Lin, C. A hierarchical neural model of data prefetching. In *26th ACM International Conference on Architectural Support for Programming Languages and Operating Systems* (2021), ASPLOS '21.

[35] Somogyi, S., Wenisch, T., Ferdman, M., and Falsafi, B. Spatial memory streaming. *Journal of Instruction-Level Parallelism (JILP) 13* (2011), 1–26.

[36] Somogyi, S., Wenisch, T. F., Ailamaki, A., Falsafi, B., and Moshovos, A. Spatial memory streaming. In *33rd Annual International Symposium on Computer Architecture* (2006), ISCA '06.

[37] Srinath, S., Mutlu, O., Kim, H., and Patt, Y. N. Feedback directed prefetching: Improving the performance and bandwidth-efficiency of hardware prefetchers. In *IEEE 13th International Symposium on High Performance Computer Architecture* (2007), HPCA '07.

[38] Wu, H., Nathella, K., Pabst, M., Sunwoo, D., Jain, A., and Lin, C. Practical temporal prefetching with compressed on-chip metadata. *IEEE Transactions on Computers 71*, 11 (2022).

[39] Wu, H., Nathella, K., Pusdesris, J., Sunwoo, D., Jain, A., and Lin, C. Temporal prefetching without the off-chip metadata. In *52nd Annual IEEE/ACM International Symposium on Microarchitecture* (2019), MICRO-52.

[40] Wu, H., Nathella, K., Sunwoo, D., Jain, A., and Lin, C. Efficient metadata management for irregular data prefetching. In *46th International Symposium on Computer Architecture* (2019), ISCA '19.